MODULE 11 · DOMAIN 4 - TERRAFORM CONFIGURATION

Dependencies and Lifecycle Rules

depends_on, create_before_destroy, and the three other rules that change how Terraform behaves.

DOMAIN 4 of 8 COURSE WEIGHT 24% MODULE 11 of 27

admin@sthithapragna.com

Lifecycle settings do not change what Terraform tells the provider. They change **how Terraform builds and walks the graph**.

Implicit dependencies, restated

- Referencing one resource's attribute inside another **creates the dependency automatically**.
- This is how nearly all ordering should be expressed. It is precise, self-documenting, and impossible to forget.
- The dependency is **exactly as narrow as the reference**, so the graph stays accurate.
- Module seven's failure: a hardcoded identifier gives the right value and **no dependency at all**.
- Everything else in this module exists for cases references cannot express.

> ON THE EXAM
>
> Reach for depends_on only after establishing that no reference can express the relationship. That is the order of operations.

depends_on

- **depends_on** declares a dependency Terraform cannot infer, because no value passes between the two.
- The classic case: a resource needs **an IAM policy or a permission** that must exist first, but references nothing from it.
- It takes a **list of resource or module addresses**, not arbitrary expressions.
- It is available on **resources, modules and data sources**, and behaves differently on the last of those.
- **It does not force replacement.** A changed dependency is ordered before, not propagated into.

> ON THE EXAM
>
> depends_on adds an edge to the graph. It never passes a value, and it never causes the dependent resource to change.

Implicit or explicit?

IMPLICIT, BY REFERENCE

Almost always this

- Created by using a value
- Narrow and accurate
- Self-documenting in the code
- Keeps plan output informative

EXPLICIT, WITH DEPENDS_ON

Only when nothing is passed

- Declared as a list of addresses
- Coarse: the whole object, not an attribute
- Needs a comment explaining why
- Can make downstream values unknown

THE TIEBREAKER

On a data source, depends_on with a changing dependency defers the read to apply. Module seven showed what that costs.

The lifecycle block

- A nested **lifecycle** block inside a resource. It is a meta-argument, available on every resource type.
- Four behaviour rules: **create_before_destroy, prevent_destroy, ignore_changes, replace_triggered_by**.
- Plus **precondition** and **postcondition**, which are custom conditions and belong to module twelve.
- **It cannot be generated by a dynamic block.** Module ten explained exactly why.
- Rules can be combined, and combinations interact in ways worth knowing.

> ON THE EXAM
>
> The lifecycle block changes Terraform's behaviour, not the resource's configuration. Nothing in it reaches the provider API.

Literal values only

- **Lifecycle settings shape how Terraform builds and traverses the graph.**
- **So only literal values are allowed.** Processing happens too early to evaluate expressions.
- You **cannot** set prevent_destroy from a variable, or drive create_before_destroy from an input.
- Third appearance of one principle: **what the engine needs early cannot come from what runs late**.
- The others: **plan-time keys**, and **dynamic blocks not generating meta-arguments**.

> ON THE EXAM
>
> A module cannot offer 'protect this resource' as a variable. Callers who need that must write the lifecycle block themselves.

create_before_destroy

- By default, a replacement **destroys the old object first**, then creates the new one. That means an interruption.
- **create_before_destroy reverses the order**: build the replacement, then remove the original.
- The reason to use it: **avoiding downtime** on load balancers, certificates, addresses and instances.
- **It is opt-in,** and deliberately so. Terraform cannot know whether reversing the order is safe for a given resource type.
- **Destroy provisioners do not run** when create_before_destroy is set.

> ON THE EXAM
>
> This is objective 4f by name. Expect it as the answer whenever a scenario says a replacement must not cause an outage.

Why it is opt-in: the name collision

- **Many remote object types have unique name requirements**, and two cannot exist at once.
- So creating the replacement first **fails**, because the original still holds the name.
- Some resource types offer a **name prefix** option that appends a random suffix, which resolves it.
- **Terraform cannot activate those features for you.** You must know the constraints of the resource type.
- Other constraints exist too: quotas, port bindings, and anything else that admits only one holder.

> ON THE EXAM
>
> create_before_destroy plus a fixed unique name is a failed apply, not a safer one. Check the resource type first.

It propagates to dependencies

- **create_before_destroy propagates to all of a resource's dependencies.**
- Set it on A, and if A depends on B, it is **enabled on B implicitly**.
- **You cannot override it back to false on B.** That would imply a cycle.
- The logic is unavoidable: A cannot be created before its dependency exists.
- So it can change replacement behaviour **for things you did not name**.

> ON THE EXAM
>
> The propagation is not a courtesy. Without it the ordering would be impossible, which is why Terraform refuses to let you disable it.

prevent_destroy

- **prevent_destroy** makes Terraform **error rather than proceed** with any plan that would destroy the resource.
- The intent: a guard on databases, state buckets and anything else where deletion is unrecoverable.
- **It blocks replacement too**, because replacement destroys the existing object.
- **create_before_destroy does not bypass it.** That changes ordering, not whether a destroy happens.
- It must be a **literal true**. It cannot be driven by a variable.

> ON THE EXAM
>
> Any plan containing a destroy of that resource fails, including a replacement forced by an unrelated attribute change.

What prevent_destroy misses

- **Remove the resource block and the protection goes with it.** The apply destroys it.
- The rule lives **in the configuration**. Deleting the configuration deletes the rule.
- Documented behaviour, not a bug, and the most misunderstood thing about it.
- To keep the object and stop managing it, **remove it from state**. Module eighteen.
- Real protection lives in the **provider**: deletion flags, and permissions.

> ON THE EXAM
>
> prevent_destroy guards against a careless change. It cannot guard against deleting the block, because the guard was in the block.

ignore_changes

- Takes a **list of attribute names** that Terraform should stop comparing against configuration.
- The use case: an **external system legitimately modifies the resource**, such as autoscaling or a policy engine.
- Elements of maps and lists can be named with index notation, so you can ignore one tag rather than all of them.
- **Only attributes defined by the resource type.** It cannot be applied to itself or to any other meta-argument.
- **Ignoring an attribute does not prevent replacement** when a different argument still forces one.

> ON THE EXAM
>
> This is the one lifecycle rule that hides information from you. Every use of it should carry a comment saying why.

ignore_changes = all

- The keyword **all** ignores every attribute, rather than a named list.
- **Terraform can then create and destroy the object, and will never propose an update to it.**
- So your configuration **stops describing** the running resource, and nobody is told.
- Occasionally correct: an object handed to another system entirely after creation.
- Usually a mistake, and a durable one, because the plan will stay clean while reality drifts away.

> ON THE EXAM
>
> You keep the create and destroy stages and give up the update stage. That is a big trade to make in one word.

replace_triggered_by

- Added in **Terraform 1.2**. Replaces this resource when something else changes.
- Takes a list of expressions referencing **managed resources, instances, or instance attributes**.
- The use case: a resource that must be rebuilt when its **configuration source** changes, with no argument linking them.
- **depends_on cannot do this.** That orders operations; it never propagates a replacement.
- Inside count or for_each, you can use **count.index or each.key** to reference the matching instance.

> ON THE EXAM
>
> depends_on says 'after'. replace_triggered_by says 'again'. Those are different requests, and questions pair them.

How the rules interact

COMBINATION	WHAT HAPPENS
prevent_destroy with create_before_destroy	Still blocked. Ordering changed; a destroy still occurs
create_before_destroy on a dependency	Enabled implicitly, and cannot be set back to false
ignore_changes with a forced replacement	Replacement still happens, if another argument forces it
ignore_changes with replace_triggered_by	The trigger still fires. Ignoring is not immunity
create_before_destroy with destroy provisioners	The destroy provisioners do not run at all

ON THE EXAM

These rules are not independent switches. Combinations appear in questions precisely because people assume they are.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
A replacement must not cause an outage	create_before_destroy
The apply failed because the name is taken	create_before_destroy on a uniquely-named object
An autoscaler keeps fighting Terraform	ignore_changes on that specific attribute
Plans are clean but the resource has drifted	ignore_changes = all, hiding the difference
Rebuild this whenever that changes	replace_triggered_by, not depends_on
The database was destroyed despite protection	The resource block was deleted, taking the rule with it

ON THE EXAM

Ask what is being requested: an order, a guard, a blind spot, or a rebuild. Each maps to exactly one rule.

A production database carries prevent_destroy = true, added two years ago after an incident. During a repository cleanup, an engineer removes what she believes is an unused module call, which included that database's resource block.

The plan shows the database will be destroyed. She is surprised, having expected prevent_destroy to block it.

What is the correct explanation?

A prevent_destroy only applies to terraform destroy, not to a normal apply

B Removing the resource block removes the rule with it, so the destroy proceeds

C prevent_destroy expired, because lifecycle rules are only honoured for one year

D The plan is a preview only. Apply would fail with a prevent_destroy error

◆ PAUSE AND DECIDE

What is the correct explanation?

A Only blocks destroy command

False. prevent_destroy blocks any plan containing a destroy of that resource, including a replacement forced by an attribute change.

B The rule went with the block

Correct, and documented. HashiCorp states Terraform destroys the infrastructure if you remove the configuration, even when prevent_destroy was enabled.

C It expired

Invents a mechanism. Lifecycle rules have no lifetime; they exist exactly as long as the block containing them.

D Apply would fail

The dangerous belief. The plan is accurate, and the apply would carry it out. There is no second safety net behind the plan.

THE SIGNAL

The guard was inside the thing that was deleted. To keep the object while stopping management, remove it from state instead.

A load balancer is replaced whenever its subnet configuration changes, causing about ninety seconds of downtime. An engineer adds create_before_destroy = true to eliminate it.

The next apply fails partway through, reporting that a resource with the requested name already exists. The load balancer has a fixed name set from a variable.

What is the cause?

A create_before_destroy requires the resource to also set prevent_destroy = false

B The replacement cannot be created while the original still holds the unique name

C create_before_destroy is unsupported for load balancers in most providers

D The apply needs -parallelism=1 so the destroy completes before the create begins

◆ PAUSE AND DECIDE

What is the cause?

A Needs prevent_destroy = false

Invents a dependency between two unrelated rules. Absent prevent_destroy is simply absent; it does not need declaring as false.

B The name is still held

Correct. This is exactly why HashiCorp made the setting opt-in: many object types have unique name requirements and cannot coexist.

C Unsupported for load balancers

It is a Terraform-side ordering rule available on every resource type. The constraint is the naming, not the resource category.

D Set parallelism to 1

Reverts to destroy-then-create sequencing, which is the downtime she was removing. It undoes the change rather than fixing it.

THE SIGNAL

The fix is a name prefix with a random suffix, where the resource type offers one. Terraform cannot enable that for you.

> Eighteen months ago, an engineer added ignore_changes = all to a compute cluster resource, to stop Terraform reverting changes an operations team was making by hand. It worked, and plans have been clean ever since.
>
> A new engineer runs a plan expecting to change the instance type. The plan reports no changes at all.

What should the team understand and do?

A State is stale. Run terraform apply -refresh-only to reconcile it

B ignore_changes = all means Terraform will never propose an update. Narrow it to the attributes actually managed elsewhere

C The instance type is a computed attribute and cannot be changed by Terraform

D Add replace_triggered_by so the change is applied as a replacement instead

◆ PAUSE AND DECIDE

What should the team understand and do?

A Refresh-only

Refresh-only records reality into state. It changes nothing about the rule that suppresses every proposed update.

B Narrow the ignore list

Correct. all keeps create and destroy and removes update entirely. Naming the specific contested attributes restores management of everything else.

C Computed attribute

Instance type is an ordinary configurable argument. Nothing about it is provider-computed.

D replace_triggered_by

Forces replacement when something else changes. It neither restores update behaviour nor addresses why the update was suppressed.

THE SIGNAL

The original fix was correct in intent and far too wide. Eighteen months of clean plans is the symptom, not the reassurance.

> An application instance reads its configuration from an object in a storage bucket at boot time only. When the object's contents change, the instance keeps running the old configuration until it is rebuilt.
>
> The team wants the instance rebuilt automatically whenever that object changes. No argument on the instance references the object.

Which mechanism does this?

A depends_on referencing the storage object

B replace_triggered_by referencing the storage object

C create_before_destroy on the instance

D ignore_changes on the instance's user data

◆ PAUSE AND DECIDE

Which mechanism does this?

A depends_on

Orders operations only. The object would be updated before the instance is considered, and the instance would not change at all.

B replace_triggered_by

Correct. It replaces this resource when a referenced resource or attribute changes, which is precisely the requirement.

C create_before_destroy

Changes the order of a replacement that is already happening. It never causes one.

D ignore_changes

The opposite direction entirely. It suppresses reactions to change rather than creating one.

THE SIGNAL

depends_on says after. replace_triggered_by says again. Scenarios pair them precisely because they look similar in a list.

Module 11 recap

- **Prefer implicit dependencies.** depends_on is for relationships no reference can express.
- **Lifecycle settings take literal values only,** because they shape the graph before expressions are evaluated.
- **create_before_destroy avoids downtime** and is opt-in because of unique name constraints.
- **It propagates to dependencies** and cannot be disabled on them.
- **prevent_destroy dies with its block.** Delete the resource block and the infrastructure goes.
- **depends_on says after. replace_triggered_by says again.**

> ON THE EXAM
>
> ignore_changes = all keeps create and destroy and removes update. Clean plans then mean nothing at all.